

Appendix A — Surface Management

Transient interface elements such as context menus, overlays, modal dialogs, notifications, and floating tool windows are commonly implemented through specialized runtime systems that differ from ordinary interface composition.

Within a surface-based projection architecture, these elements do not require dedicated synchronization concepts. They differ primarily in lifetime, placement, and interaction behavior while remaining structurally equivalent to ordinary runtime surfaces.

Representing transient interface elements as regular surfaces preserves a uniform projection hierarchy and allows creation, visibility management, ownership, and synchronization to be handled through the same mechanisms already used throughout the runtime. Presentation-specific behavior is delegated to the browser runtime through configurable placement and interaction policies rather than introducing additional architectural abstractions

A.1 Surface-Based Popup Architecture

Popup-oriented interface elements are represented as ordinary child surfaces within the existing projection hierarchy. No distinction is made between a popup surface and any other synchronized surface beyond presentation and interaction policies.

A context menu attached to a workspace surface may therefore be represented as:

```
Root Surface
├─ Workspace Surface
│   ├── Asset Browser
│   ├── Inspector
│   └─ Context Menu Surface
```

Creation of popup surfaces uses the existing synchronization protocol without introducing additional commands. The browser runtime creates the corresponding surface instance and attaches it to the specified parent surface. Visual behavior is determined through placement policies rather than synchronization semantics.

The runtime defines a set of standard placement policies:

```
const SurfacePlacement =
{
  Cursor: "cursor",
  Center: "center",
  Anchor: "anchor",
  TopLeft: "top-left",
  TopRight: "top-right",
  BottomLeft: "bottom-left",
  BottomRight: "bottom-right"
};
```

Placement may optionally reference an anchor position:

```
{
  placement: cursor,
  anchor:
  {
    x: 640,
    y: 380
  }
}
```

```
}  
}
```

```
    return { x, y };  
}
```

The browser runtime resolves the final surface position according to the selected policy while ensuring that the projected surface remains fully visible within the current viewport.

A generic placement routine may be implemented as follows:

```
function placeSurface(width, height, policy,  
    anchor)  
{  
    let x = 0;  
    let y = 0;  
  
    switch (policy)  
    {  
        case SurfacePlacement.Cursor:  
        case SurfacePlacement.Anchor:  
  
            x = anchor.x;  
            y = anchor.y;  
            break;  
  
        case SurfacePlacement.Center:  
  
            x = (window.innerWidth - width) / 2;  
            y = (window.innerHeight - height) / 2;  
            break;  
    }  
  
    if (x + width > window.innerWidth)  
        x = window.innerWidth - width;  
  
    if (y + height > window.innerHeight)  
        y = window.innerHeight - height;  
  
    x = Math.max(0, x);  
    y = Math.max(0, y);
```

This guarantees that popup-oriented surfaces remain visible regardless of cursor position or viewport size. By centralizing placement behavior within the runtime, context menus, tooltips, notifications, dropdown menus, modal dialogs, and future popup-oriented surface types automatically inherit consistent positioning behavior without requiring individual implementations

A.3 Visibility Management

Popup-oriented surfaces are not fundamentally different from persistent surfaces. The primary distinction is whether a surface is currently projected to the user. Visibility therefore becomes a presentation concern rather than a lifecycle concern. A context menu may remain synchronized and attached to its parent surface while transitioning between visible and hidden states. This allows the runtime to treat visibility independently from ownership, placement, and synchronization.

Visibility changes may originate from server-side state transitions, local runtime policies, or event-driven context evaluation. By separating visibility from surface identity, the runtime avoids introducing dedicated popup lifecycles and preserves a consistent projection model across all surface types

A.4 Surface Ownership

Popup-oriented surfaces are typically associated with another surface or element. Ownership establishes a relationship between a parent projection and dependent

surfaces without introducing specialized synchronization semantics.

Ownership may be used by the runtime to determine visibility, focus behavior, placement anchors, and interaction routing. The ownership hierarchy remains part of the ordinary surface graph and therefore participates in synchronization through the same mechanisms used by all other projected surfaces

A.5 Event Contexts and Context-Aware Composition

Interactive interfaces frequently require behavior that depends on the current projection state. Examples include context-sensitive menus, tooltips, inspector layouts, toolbar variations, and dynamic workspace content.

Rather than introducing specialized systems for each use case, the runtime defines event contexts as reusable resources that may be associated with projected elements through pseudo-properties. Contexts are registered similarly to templates and compositions. Unlike templates and compositions, contexts are not evaluated continuously. A context is always associated with a specific event.

An element may therefore expose:

```
{  
  onClickContext: 42,  
  onHoverContext: 43  
}
```

When the corresponding event occurs, the browser runtime resolves the referenced context and evaluates it against the current projection state.

Event



Context



Projection Update

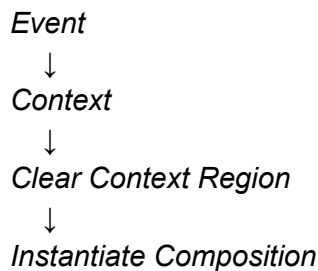
The context receives access to any of event information, element properties, selection and projection states, and may produce composition changes, property changes and optional return values consumed by the runtime.

A context menu may therefore be resolved through an event context without requiring a roundtrip to the display server, while contexts do not replace server authority. Instead, they allow the browser runtime to evaluate server-defined interaction rules against already synchronized projection data. This preserves responsiveness while maintaining authoritative application state on the server

A.6 Dynamic Composition Regions

Context evaluation may produce structural changes within an existing surface. Rather than rebuilding the entire surface hierarchy, runtimes may reserve dedicated composition regions whose contents are managed by event contexts. A generic surface may therefore consist of both persistent and dynamic regions.

The persistent region contains synchronized content whose identity remains stable throughout the lifetime of the surface. The context region contains dynamically instantiated compositions selected by context evaluation.



This approach preserves the identity of the surrounding surface while allowing context-sensitive portions of the interface to be rebuilt as needed. For example, a context menu surface may retain its position, ownership, visibility state, and synchronization identity while replacing only the currently active menu composition. This separation allows dynamic interface content to remain lightweight while preserving stable runtime state

A.7 Docking and Workspace Anchoring

Placement behavior is not limited to transient popup surfaces. Persistent interface regions such as menu bars, toolbars, status bars, side panels, and taskbars can be expressed through docking policies that extend the placement system.

The runtime may define standard docking regions to assemble a workspace:

```
const SurfaceDock =
{
  Top: "top",
  Bottom: "bottom",
  Left: "left",
  Right: "right",
  Fill: "fill"
};
```

Docked surfaces remain ordinary synchronized surfaces. They differ only in how their placement is resolved by the runtime. Because docking is expressed as a placement policy rather than a synchronization concept, future runtime extensions may implement more advanced workspace managers, docking frameworks, desktop shells, and editor environments without requiring modifications to the underlying display server protocol.

The synchronization layer remains concerned solely with projected surfaces and their relationships, while workspace-specific layout behavior is delegated entirely to the browser runtime

Appendix B — Workspace Composition

The surface model introduced mechanisms for synchronizing and projecting interface components. While individual surfaces are sufficient for simple applications, development tools typically consist of multiple cooperating surfaces organized into larger working environments.

Examples include asset management systems, property editors, scene

hierarchies, consoles, graph editors, and debugging tools.

Traditional frameworks frequently introduce specialized workspace abstractions to coordinate these components. Within a surface-based projection architecture, no additional synchronization concepts are required.

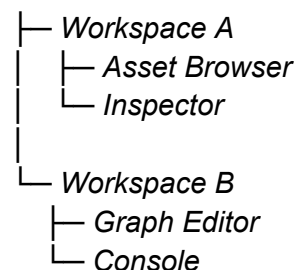
A workspace is not a specialized runtime object. A workspace is simply a collection of surfaces whose placement, visibility, focus, and ownership are coordinated through runtime policies. This preserves the uniformity of the projection model while allowing complex editor environments to be assembled from ordinary synchronized surfaces

B.1 Workspace Hierarchies

A workspace is represented as an ordinary surface that owns additional child surfaces. The workspace itself does not introduce new synchronization semantics. It participates in the projection hierarchy exactly like any other surface. Workspaces therefore remain composable.

A larger environment may contain multiple workspaces:

Root Surface



Because workspaces are ordinary surfaces, ownership, visibility, placement, and context evaluation continue to operate through the same mechanisms described in Appendix A. No distinction exists between a workspace surface and any other projected surface beyond the policies applied by the runtime

B.2 Docking and Layout Composition

Development environments commonly organize surfaces through docking systems. Within the projection architecture, docking is treated as a runtime layout policy rather than a synchronization concept.

A runtime may provide a standard set of docking regions:

```
const SurfaceDock =  
{  
  Top: "top",  
  Bottom: "bottom",  
  Left: "left",  
  Right: "right",  
  Fill: "fill"  
};
```

These regions represent a common baseline rather than a fixed limitation. Runtime implementations may introduce additional docking policies such as tab groups, floating windows, auto-hide panels, workspace groups, desktop shells, or application-specific layouts.

The browser runtime translates these placement policies into an appropriate layout system such as CSS Grid, Flexbox,

absolute positioning, or custom workspace managers.

A simple runtime implementation using CSS Grid may therefore end in a layout comparable to:

```
+-----+-----+
| Assets |           |
|       | Graph Editor |
|       |           |
+-----+-----+
|           Console           |
+-----+-----+
```

The display server remains unaware of the underlying layout implementation and continues to operate solely on surface relationships. Layout composition therefore becomes a browser runtime concern while preserving protocol simplicity

B.3 Workspace Persistence

Workspace composition frequently requires surfaces to remain available even when they are not currently visible. Destroying and recreating these surfaces whenever the user changes layouts introduces unnecessary synchronization overhead and may result in the loss of local projection state.

Instead, runtimes may temporarily detach or hide surfaces while preserving their identity. A hidden surface remains synchronized and continues to participate in the workspace hierarchy, while a detached surface temporarily leaves the active layout preserving ownership and runtime state.

Because surface identity remains unchanged, the display server can continue to manage placement, focus, visibility, and synchronization without requiring recreation of the underlying projection. This allows

complex workspaces to evolve dynamically while maintaining stable runtime identities

B.4 Workspace Switching

Many environments allow multiple surfaces to share the same visual region. Examples include tab bars, tool groups, stacked inspectors, and alternative editor views.

A tabbed interface may therefore be represented as:

Workspace

- └ *Tab Bar*
- └ *Content Region*

where the content region hosts multiple surfaces. Switching between tabs does not require introducing a specialized tab surface. Instead, the runtime adjusts visibility or active composition state.

Depending on the implementation, a runtime may either preserve hidden surfaces or dynamically replace compositions within the active region. Both approaches remain compatible with the existing surface model.

Workspace switching therefore becomes an expression of visibility and composition management rather than a separate synchronization concept

B.5 Focus, Selection and Activation

Interactive workspaces require a consistent mechanism for routing input and determining which projection currently receives user interaction. Rather than

maintaining a single global focus state, the runtime organizes focus and selection hierarchically.

Each surface maintains its own local interaction state. Within a surface, only one element may possess focus and only one hierarchy path may be selected at a time, while multiple surfaces may simultaneously maintain independent local states.

These states coexist without conflict because they belong to different surfaces. A workspace introduces an additional state layer.

Workspace

- |— *Surface A*
- |— *Surface B*
- |— *Surface C*

Only one surface is considered active at any given time. The active surface determines which local interaction state is exposed to the workspace.

Input routing therefore follows a hierarchical model:

Workspace

↓
Active Surface
↓
Focused Element

Commands such as Copy, Paste, Delete, Rename or Duplicate, are routed to the active surface rather than broadcast throughout the workspace.

State information on the other hand propagates upward through the hierarchy, while commands flow downward:

Focused Element

↓
Focused Surface
↓
Active Workspace Surface

Workspace Command

↓
Active Surface
↓
Focused Element

This bubbling model allows multiple independent interaction domains to coexist while maintaining deterministic input routing

B.6 Workspace Environments

The workspace architecture closely resembles the structure of traditional desktop environments. A desktop system typically consists of Desktop, Windows and Controls. A workspace environment follows an equivalent structure:

Workspace

↓
Surface
↓
Element

In both systems, independent containers maintain local focus, selection, and

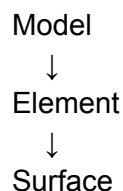
interaction state while participating in a larger activation hierarchy. The distinction lies primarily in application semantics rather than runtime architecture.

Consequently, the display server does not differentiate between editor tooling, browser-hosted operating environments, or custom workspace systems. All are expressed through coordinated collections of synchronized surfaces operating within a shared placement, ownership, visibility, focus, and activation hierarchy.

The workspace therefore serves as the bridge between individual projected surfaces and complete browser-hosted application environments

Appendix C — Composed Interface Systems

Surfaces provide identity, ownership, visibility, placement, focus management, and interaction routing while remaining independent of any specific application domain. The system introduced three fundamentally different concepts that are necessary to distinguish between, while they participate differently in the projection model.



Although these concepts are frequently merged within traditional user interface frameworks, each fulfills its own responsibility within the runtime architecture

Models

Models represent application state. They define data, relationships, and execution semantics independently from visual presentation.

Models exist independently from the browser runtime and may continue to exist even when no corresponding user interface is projected. A node graph therefore remains a node graph regardless of whether it is displayed within an editor, transmitted over a network connection, or processed by an automated build system.

The graph contains only application data and execution relationships. No visual information is required. The model remains entirely independent from presentation concerns

Elements

Elements provide visual representation of model data. Unlike surfaces, elements do not participate in the projection hierarchy as independently synchronized objects. Instead, they are lightweight structural components whose compositions are propagated from the display server, while behavior is defined by the runtime.

Elements are generally instantiated through compositions and templates within the synchronization events.

A node editor may contain thousands of nodes simultaneously. Treating every node as an independently synchronized surface

would introduce unnecessary runtime overhead while providing little architectural benefit. Nodes therefore remain elements. The runtime projects them as part of a larger composition rather than as independent surfaces

Surfaces

Surfaces represent independently managed projection domains. A surface possesses identity within the display server and participates in synchronization, ownership, visibility management, focus routing, placement policies, and context evaluation.

Unlike elements, surfaces are capable of maintaining independent interaction state without affecting neighboring surfaces. This distinction becomes particularly important in large editor environments where multiple independent interaction domains coexist simultaneously

Projection Hierarchy

The resulting projection hierarchy can therefore be described as:

Workspace
└─ Surface
 └─ Elements

Models remain external to the projection hierarchy and are referenced by projected elements:

Model
↓
Elements
↓

Surface



Workspace

The display server remains responsible only for synchronizing surfaces and their associated state. Element composition, rendering, and interaction behavior are delegated to the browser runtime

Architectural Consequences

This separation produces several important properties:

1. Synchronization complexity remains proportional to the number of active surfaces rather than the number of visible elements.

A graph editor containing thousands of nodes continues to operate as a single synchronized surface

2. Models remain entirely independent from presentation concerns.

The same graph may therefore be projected through a visual editor, processed through automated tooling, or executed on a remote server without modification

3. Surfaces become reusable containers capable of hosting arbitrary element compositions.

The display server does not need to understand whether a surface contains buttons, nodes, property grids, text editors, streamed video, or future interface constructs. It synchronizes only projection domains while the browser runtime

remains responsible for rendering their contents

This distinction between models, elements, and surfaces forms the foundation for all remaining systems discussed

C.1 Node-Based Graph Editor

Node-based editors are among the most demanding interactive systems commonly found within development environments. Unlike traditional forms, lists, or property grids, graph editors require continuous interaction between projected elements, dynamic relationships, viewport transformations, and real-time visual updates.

Despite their complexity, graph editors can be implemented entirely through standard browser technologies without introducing specialized rendering frameworks. Within the projection architecture, a graph editor is represented as a single synchronized surface responsible for projecting graph-related elements and maintaining viewport state

Graph Surface Architecture

A graph editor is represented as an ordinary surface. The graph itself remains part of the application model.

```
const Graph =  
{  
  nodes: [],  
  connectors: [],  
  connections: []  
};
```

Nodes, connectors, and connections are not surfaces. They are projected elements belonging to the graph surface.

The display server synchronizes only the graph surface while the browser runtime projects the graph contents. This ensures synchronization complexity remains proportional to the number of active surfaces rather than the number of visible graph elements.

A graph containing thousands of nodes therefore remains a single synchronized surface

Node Projection

Nodes are projected through ordinary HTML templates. The browser runtime creates a corresponding DOM representation:

```
<div class="node" id=1270011>  
  <div class="title">Multiply</div>  
  
  <div class="inputs">  
    ...  
  </div>  
  
  <div class="outputs">  
    ...  
  </div>  
</div>
```

Nodes are positioned through absolute placement within the graph surface via CSS styles:

`.node`

```
{
  position: absolute;
}
```

Projection simply applies the synchronized coordinates:

```
element.style.left = `${node.x}px`;
element.style.top = `${node.y}px`;
```

Because nodes are ordinary HTML elements, they automatically benefit from browser layout, text rendering, styling, event handling, and context evaluation

Connectors and Connections

Connections are represented separately from nodes. A connector is an independently addressable graph element. Connections therefore reference connector identities:

```
const Connection =
{
  id: 42,

  source: 100,
  target: 200
};
```

This design intentionally avoids direct node references. Connections describe relationships between endpoints rather than ownership relationships between nodes.

The model therefore becomes:

```
Node
  ↓
Connector
  ↓
Connection
```

This allows future support for dynamic pins, reroute nodes, subgraphs, node collapsing, and other advanced graph constructs without modifying connection semantics

Connection Projection

Connections are projected through a dedicated SVG layer in the browser runtime:

```
<div class="graph-surface">

  <svg class="connections"></svg>

  <div class="nodes"></div>

</div>
```

The graph surface therefore contains two rendering layers: the SVG Connection Layer and the HTML Node Layer. Nodes therefore remain ordinary HTML elements while connections are represented through SVG paths.

Each connection owns a corresponding SVG path. The browser runtime maintains a lookup table allowing projected paths to be located efficiently. Connections remain visually behind nodes:

```
.connections
{
```

```

    z-index: 0;
  }
  .nodes
  {
    z-index: 1;
  }

```

Connector Position Resolution

Connection geometry is derived from projected connector positions. The browser provides access to the final layout position of any element through `getBoundingClientRect`.

The center position of a connector can therefore be calculated as:

```

function getConnectorCenter(element)
{
  const rect = element.
    getBoundingClientRect();

  const graphRect = graphContent.
    getBoundingClientRect();

  return {
    x:
      rect.left -
      graphRect.left +
      rect.width * 0.5,

    y:
      rect.top -
      graphRect.top +
      rect.height * 0.5
  };
}

```

Using projected positions rather than model coordinates ensures that connection

rendering automatically respects styling, padding, borders, fonts, and template-specific layout variations

Bezier Connection Rendering

Connections are rendered as cubic Bézier curves:

```

function buildConnectionPath(start, end)
{
  const dx =
    (end.x - start.x) * 0.5;

  return `
    M ${start.x} ${start.y}
    C ${start.x + dx} ${start.y}
      ${end.x - dx} ${end.y}
      ${end.x} ${end.y}
  `;
}

```

Projection updates the associated SVG path:

```

function updateConnection(connection)
{
  const start =
    getConnectorCenter(
      connectorElements.get(
        connection.source));

  const end =
    getConnectorCenter(
      connectorElements.get(
        connection.target));

  const path =
    buildConnectionPath(start, end);

  connectionElements

```

```

    .get(connection.id)
    .setAttribute("d", path);
}

```

The resulting curve automatically adjusts to changes in node position and connector layout

Node Movement

Nodes may be repositioned through pointer interaction:

```

function onNodeDrag(node, dx, dy)
{
    node.x += dx;
    node.y += dy;

    updateNodeElement(node);
    invalidateNode(node);
}

```

Updating the node projection does not immediately rebuild connection geometry. Instead, affected connections are marked as dirty

Incremental Connection Updates

Efficient graph editing requires selective rather than global connection rebuilding. The runtime therefore maintains a bidirectional lookup. When a connector position changes, only affected connections are invalidated:

```

const dirtyConnections =
    new Set();

```

```

function invalidateNode(node)
{
    for(const connectorId of node.connectors)
    {
        const connections =
            connectorConnections[
                connectorId];

        for(const connectionId of connections)
        {
            dirtyConnections
                .add(connectionId);
        }
    }
}

```

Because the dirty list is implemented as a set, a connection may be referenced by multiple nodes without being rebuilt multiple times. If multiple nodes move during the same update cycle, their Connections still appear only once within the dirty set. Connection geometry is rebuilt once per update tick.

This guarantees that each connection is recalculated at most once per tick regardless of how many graph elements participate in the update

Viewport Transformations

Graph editors commonly support panning and zooming. Rather than modifying node coordinates, the runtime transforms the graph projection itself. The graph content container is transformed through CSS:

```

.graph-content
{
    transform-origin: 0 0;
}

```

```
graphContent.style.transform =
`
  translate(
    ${view.x}px,
    ${view.y}px
  )
  scale(${view.zoom})
`;
```

This causes nodes and connections to move and scale together. The underlying graph data remains unchanged

Zoom and Pan Interaction

Panning modifies the viewport offset:

```
view.x += mouseDx;
view.y += mouseDy;
```

Zooming adjusts the scale factor:

```
graph.addEventListener(
  "wheel",
  e =>
  {
    const delta =
      e.deltaY > 0
      ? 0.9
      : 1.1;

    view.zoom *= delta;
    updateTransform();
  });
```

Because the viewport transformation is applied to the graph container rather than

individual nodes, no connection rebuilding is required during pan or zoom operations. The browser automatically applies the transformation to both rendering layers

C.2 Streamed Frame Surfaces

Not all applications that participate in a development environment can be projected through ordinary browser-native elements. Many systems generate visual output through rendering technologies that are not directly compatible with the browser execution environment. For example Game engines, simulation systems or CAD applications.

While portions of these applications may expose browser-native interfaces, their primary visual output often exists only as rendered frames. Reimplementing such systems using browser technologies would significantly increase development complexity while providing little architectural benefit.

Within the projection architecture, these applications are instead represented as streamed frame surfaces. A streamed surface behaves identically to any other surface within the display server hierarchy. The only distinction is that visual content originates from an external frame producer rather than local browser elements

Frame Surfaces

A frame surface is an ordinary surface whose content is supplied through a continuous frame stream. Unlike ordinary surfaces, a frame surface projects live content from an external source directly to the browser runtime.

The display server does not interpret the visual contents of the stream. Instead, the surface remains responsible only for visibility, focus, ownership and placement. All rendering logic remains external to the browser runtime. This preserves architectural consistency across all surface types

Streamtransport

Frame transmission requires low latency and continuous updates. Unlike traditional media delivery systems, development tools require immediate visual feedback and bidirectional communication. Frame delivery therefore prioritizes:

- Low Latency
- Continuous Updates
- Hardware Decoding
- Adaptive Bandwidth Usage
- Bidirectional Communication

WebRTC provides these characteristics while remaining natively supported by modern browsers. The projection architecture therefore treats WebRTC as a transport mechanism rather than as a user interface technology:

Frame Producer



WebRTC



Browser Surface

The resulting stream becomes a visual source that can be attached to any surface instance

WebRTC Projection

The browser runtime projects incoming frames through a standard video element:

```
<div class="viewport-surface">
```

```
  <video
    autoplay
    playsinline>
  </video>
```

```
</div>
```

Once a connection has been established, the incoming media stream is attached to the projected surface:

```
peerConnection.ontrack =
  event =>
  {
    video.srcObject =
      event.streams[0];
  };
```

The browser then becomes responsible for decoding and presenting incoming frames. No additional rendering infrastructure is required.

Because the frame stream is projected through a normal surface, placement, visibility management, docking, tab systems, and workspace organization continue to operate without modification. The synchronization protocol remains unchanged

Input Routing

Visual output alone is insufficient for interactive applications. Input events generated within the browser must be routed back to the originating application. WebRTC data channels provide a lightweight bidirectional communication mechanism that complements frame transport.

Mouse movement may be forwarded as:

```
video.addEventListener(
  "mousemove",
  e =>
  {
    dataChannel.send(
      JSON.stringify(
        {
          type: "mouseMove",
          x: e.offsetX,
          y: e.offsetY
        }
      ));
  });
```

Mouse buttons may be transmitted similarly:

```
video.addEventListener(
  "mousedown",
  e =>
  {
    dataChannel.send(
      JSON.stringify(
        {
          type: "mouseDown",
          button: e.button,
          x: e.offsetX,
          y: e.offsetY
        }
      ));
  });
```

```
});
```

Keyboard input may be forwarded through the same channel:

```
window.addEventListener(
  "keydown",
  e =>
  {
    dataChannel.send(
      JSON.stringify(
        {
          type: "keyDown",
          key: e.key
        }
      ));
  });
```

On the producing side, messages are interpreted and converted into native input events. The browser therefore acts as an input relay while application authority remains external

Synchronization Boundaries

A streamed frame surface differs fundamentally from previously discussed projection systems. Node editors, forms, menus, and property grids synchronize application state. Frame surfaces don't.

The browser possesses no understanding of the rendered contents. It receives only a sequence of frames:

```
Application State
↓
Frame Generation
↓
WebRTC Stream
```

↓
Browser

The browser cannot directly modify application state. Instead, all interaction occurs through routed input events which are interpreted by the originating application.

This separation allows arbitrary non-browser applications to participate within the surface hierarchy without requiring browser-specific implementations. Any system capable of generating frames and receiving input events can therefore be integrated into the projection architecture while remaining entirely independent from the browser runtime

C.3 Virtualized Projection

Large structured datasets such as asset browsers, entity hierarchies, file explorers, logs, and search results cannot be projected efficiently by rendering all elements simultaneously into the DOM.

The limitation is not primarily data throughput, but projection cost. Even when the underlying model is small in memory representation, rendering large numbers of interactive DOM nodes introduces layout, style recalculation, and interaction overhead that scales poorly.

Virtualized projection addresses this by decoupling the size of the model from the number of simultaneously rendered elements

Surface-Backed Virtualization Model

Virtualization is not treated as a global policy but as an internal projection mechanism of a surface implementation. A virtualized surface maintains a bounded projection cache that represents only a subset of the full dataset:

Surface
↓
Projection Cache
↓
DOM Elements

The full dataset remains authoritative on the server, while the runtime cache is continuously synchronized through incremental updates

Projection Cache and Visible Window

Each virtualized surface maintains a local projection cache that stores a bounded subset of the server-authoritative dataset. This cache is not a derivation of the DOM but an intermediate representation between model and rendering.

The visible window defines which portion of the cache is actively projected. Only elements within the visible window are instantiated as DOM nodes

Runtime Window Expansion

To avoid excessive synchronization and scrolling artifacts, the runtime does not request exactly the visible range but a

slightly expanded window. This ensures stable scrolling behavior and reduces the frequency of cache invalidation

Incremental Synchronization

The server only transmits deltas to the model state. The client applies these deltas to the projection cache without affecting the visible window unless required. Projection updates are independent of synchronization updates

Viewport-Driven Projection

The visible window is controlled entirely by the client runtime based on user interaction. Scroll and layout changes update only local projection state. The server is not queried for scroll operations, only for data changes.

DOM updates are batched to avoid layout thrashing. Rendering only instantiates elements within the current window

Surface Ownership of Cache

The projection cache is owned by the surface instance. It is neither global nor shared across unrelated surfaces.

This ensures isolation between different projections of the same dataset. Multiple surfaces may observe the same server model but maintain independent projection states

Consistency and Authority

The server remains the sole authority over data correctness. The client projection cache is a bounded, disposable representation used solely for rendering efficiency.

No modification to the projection cache is considered authoritative unless confirmed by a server-side delta

C.4 Accessible Projection

Accessibility is often treated as a browser-specific concern implemented independently from the application model. Within a projection-based architecture, accessibility information can instead be synchronized alongside visual state and become part of the projected representation itself.

This approach ensures that semantic information remains consistent across all clients while reducing the amount of accessibility-specific logic required within individual projections. Rather than deriving accessibility information from visual appearance, the server explicitly describes the semantic meaning of projected elements

Accessibility Metadata

Accessibility information is represented as projection metadata associated with synchronized elements. The runtime translates this metadata into browser-native accessibility mechanisms:

```
<button
  aria-label="Build Project">
  Build
</button>
```

The same approach applies to more complex structures. Accessibility semantics therefore originate from synchronized state

rather than being inferred from visual composition

Keyboard Navigation

Accessible interaction requires deterministic keyboard navigation across projected surfaces. The focus model introduced previously provides a natural foundation for keyboard accessibility.

At any point in time, a workspace maintains a single active surface while each surface maintains a single focused element. Keyboard input is routed through this hierarchy, ensuring predictable navigation behavior regardless of the complexity of the projected interface.

This allows browser-native focus handling, shortcut processing, and accessibility navigation to operate consistently across all surface implementations

Alternative Input Systems

Accessibility metadata is not limited to screen readers. Because semantic information is synchronized independently of visual representation, the same metadata may be consumed by a wide range of systems. Voice control systems, automation frameworks and testing infrastructures can make use of metadata in order to operate on the browser runtime.

These systems interact with semantic descriptions rather than visual appearance. As a result, accessibility becomes a property of the synchronized projection model itself rather than a browser-specific enhancement

D. Browser-Hosted Environments

The architecture presented does not target a specific application category. Instead, higher-level systems emerge from the composition of surfaces, workspaces, projection mechanisms, and synchronization models. When combined, the illustrated concepts provide the foundation for significantly larger systems without introducing additional architectural abstractions.

Such systems are not treated as specialized products within the architecture. They emerge naturally from the composition of previously introduced primitives

D.1 Environment Composition

A development environment is represented as a workspace composed of ordinary surfaces. No distinction exists between editor windows, tools, panels, or viewports. Each component participates in the same projection hierarchy and follows the same synchronization model.

The workspace itself provides organization, focus management, docking behavior, visibility control, and persistence while individual surfaces remain responsible for their projected content. This allows new tools to be introduced without modifying the underlying workspace architecture

D.2 Distributed Tool Integration

The projection model intentionally separates visual representation from execution. A projected tool does not need to execute within the browser runtime. Some tools may expose browser-native interfaces

while others may be represented through streamed frame surfaces.

Because the workspace interacts only with surfaces, the origin of the tool becomes largely irrelevant. Local services, remote services, browser-native applications, and streamed applications participate through the same projection mechanisms

D.3 Persistent Workspaces

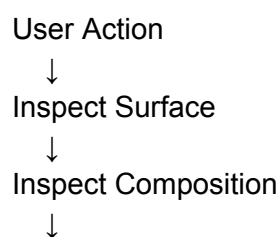
Workspace composition relies on persistent surface instances. Visibility and lifetime are treated as separate concerns. Switching tabs, changing layouts, or moving between workspaces does not require surfaces to be recreated.

Instead, surfaces remain synchronized while their visibility state changes. This allows expensive projections, streamed applications, graph editors, and large virtualized datasets to remain available without repeated initialization.

Persistence therefore becomes a property of the workspace itself rather than an implementation detail of individual tools

D.4 Runtime Customization

Because projected interfaces are defined through synchronized composition structures, they may optionally be inspected and modified at runtime. A runtime customization workflow may resemble:



Modify Projection

Such mechanisms allow users to experiment with layouts, extend existing interfaces, or construct entirely new workflows without modifying the underlying application.

The architecture does not require runtime customization but naturally supports it through the same projection model used for ordinary synchronization

D.5 Collaborative Projection

Collaboration emerges naturally from the server-authoritative synchronization model. All clients observe the same authoritative projection state while maintaining independent presentation and interaction layers.

Changes performed by one participant are propagated through the synchronization system and become visible to other connected clients.

Because collaboration operates at the projection layer, no distinction is required between single-user and multi-user environments. Both are represented through the same synchronization mechanisms

D.6 Architectural Conclusions

The architecture intentionally introduces no distinction between different tools and environments. All systems participate through the same surface hierarchy, projection mechanisms, synchronization model, and workspace structure. As a result, systems commonly described as:

- Remote IDE
- Cloud Development Environment
- Collaborative Workspace
- Web Operating Systems

may be viewed as different compositions of the same underlying architecture rather than independent categories requiring specialized infrastructure.

The display server acts as the unifying layer between execution environments and user-facing projections, allowing browser-native and non-browser-native systems to coexist within a single workspace model, whether running on a local machine or hosted as a cloud service.

Rather than defining a specific product category, the architecture defines a generalized projection framework capable of hosting a broad spectrum of interactive applications through a consistent and extensible surface model